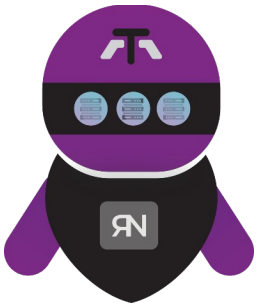

Reflection

ADMINISTRATOR & OPERATIONS MANUAL

PART II · VERSION 1.0

Configuration reference · Extensions · Firewall control
Production management · Daily operations · Troubleshooting
Upgrade & rollback · Handover acceptance record

9 Configuration reference

9.1 conf/reflect.json

Table 3. Reflector configuration keys.

Key	Meaning
`reflected_host`, `reflected_port`	Origin address and port, reachable by the reflector.
`mirror_host`, `mirror_port`	Mirror TCP control/data endpoint.
`mirror_hostudpv4`, `mirror_hostudpv6`	UDP mirror addresses.
`mirror_heartbeat_tcp`, `mirror_heartbeat_udp`	Mirror heartbeat ports.
`mirror_service_host`, `mirror_service_port`	TCP public bind address and port created on the mirror.
`mirror_service_host_udpv4`, `mirror_service_host_udpv6`	UDP public bind addresses.
`mirror_force_listener`	Replaces an existing listener for the mapping when true; avoid sharing a published port between reflectors.
`pool_size`	Idle TCP channel count / per-mapping concurrency ceiling.
`mirror_connect_timeout`	Control/UDP connection timeout in milliseconds.
`heartbeat_check_interval`	Heartbeat interval in milliseconds.
`mirror_heartbeats_lost_for_exit`	Consecutive failed beats tolerated before exit; supervisor should restart it.
`dhcp_check`, `dhcp_check_interval`	Exits when non-internal network interface addresses change; enable when a supervisor will restart after address changes.
`reflector_channel_check_wait`	Exit if the TCP channel pool is empty longer than this many milliseconds.
`keep_alive`	TCP keepalive delay in milliseconds; use -1 to skip.
`ipv6enabled`	Enables UDP IPv6 socket use.
`udp_cleanup_interval`	Idle UDP mapping cleanup time in milliseconds; -1 means no cleanup.

Key	Meaning
logfile	TCP reflector log filename below logs/.

9.2 conf/mirror.json

Table 4. Mirror configuration keys.

Key	Meaning
host, **port**	TCP mirror bind address and Reflection control/data port.
hostudpv4, **hostudpv6**	UDP bind addresses.
heartbeat_tcp, **heartbeat_udp**	Heartbeat server ports.
keep_alive, **timeout**	TCP channel keepalive and idle timeout.
udp_cleanup_interval	Idle UDP client mapping cleanup period.
ipv6enabled	Whether mirror UDP also binds IPv6.
firewall_*	Optional admission-control settings; see next section.

9.3 Worker and logging configuration

cluster_reflector.json and cluster_mirror.json set workers; 0 means use CPU count, subject to min_workers. More mirror workers add their worker index times offset_window to mirror, heartbeat, and firewall ports. In a multi-worker deployment, open and document each offset port; keep workers: 1 for a simple client installation.

log.json controls debug logging, synchronous writing, and the nominal max_log_mb. Logs are newline-delimited JSON in logs/. Rotate and retain them with the operating system's log facility; do not rely solely on the application's maximum-size setting.

10 Optional extensions and data handling

`conf/extensions.json` defines an ordered extension chain. The supplied default enables both traffic recorder and traffic logger.

Table 5. Extensions supplied with Reflection.

Extension	Configuration	Effect	Handover advice
trafficlogger	`trafficlogger.json`	Writes metadata and payload chunks to `logs/traffic.log`; payloads are Base64 by default.	Disable in production unless it is required and retention/access controls are approved.
trafficrecorder	`trafficrecorder.json`	Writes raw inbound/outbound payload streams to `logs/capture.log.in` and .out.	Treat as sensitive capture data; normally disable.
aes	`aes.json`	Adds framed AES payload encryption for TCP only.	Add only after both ends use matching extension order/key and an end-to-end test succeeds.

Set both logger/recorder enabled values to false to stop payload capture. Ensure the `logs/` directory exists before enabling extensions.

> **NOTE**
> Extension order matters: outbound processing runs in reverse order, so both ends must have the same list/order.

To enable aes extension add `{"extension": "aes"}` in `extension.json`

11 Optional firewall control

When `firewall_enabled_tcp` or `firewall_enabled_udp` is true, a newly created mirror listener denies traffic unless the source IP is explicitly allowed or passes the configured geography rules.

> **WARNING**
> The firewall control port is powerful and must be network-restricted.

11.1 Firewall administration commands

These commands manage IP access to client-facing listeners on the mirror. Set `firewall_enabled_tcp: true` on the mirror and restrict the firewall-control port to trusted administrators.

```
node lib/firewall.js open CLIENT_IP PUBLISHED_PORT MIRROR_HOST FIREWALL_CONTROL_PORT
[sfw_mode|heartbeat_mode]
```

Allows `CLIENT_IP` to connect to `PUBLISHED_PORT`. Use `all` instead of `PUBLISHED_PORT` to target every active listener.

- `sfw_mode`: automatically revokes access when that client's TCP session ends.
- `heartbeat_mode`: keeps access only while periodic heartbeat commands continue.

```
node lib/firewall.js close CLIENT_IP PUBLISHED_PORT MIRROR_HOST FIREWALL_CONTROL_PORT
```

Revokes `CLIENT_IP`'s access to `PUBLISHED_PORT` and disconnects that IP's currently active sessions. Use `all` to revoke it from every active listener.

```
node lib/firewall.js heartbeat CLIENT_IP PUBLISHED_PORT MIRROR_HOST
FIREWALL_CONTROL_PORT
```

Refreshes an existing rule created with `heartbeat_mode`. It does not create or reopen a rule. If these heartbeats stop, the mirror removes the allowance after `firewall_timeout_for_heartbeat_mode_autoclose`.

```
node lib/firewall.js check CLIENT_IP PUBLISHED_PORT MIRROR_HOST FIREWALL_CONTROL_PORT
```

Checks whether `CLIENT_IP` would currently be allowed to access `PUBLISHED_PORT`, considering explicit firewall rules and configured geofencing.

Example:

```
node lib/firewall.js open 203.0.113.10 9898 mirror.example.com 2003 heartbeat_mode
node lib/firewall.js heartbeat 203.0.113.10 9898 mirror.example.com 2003
node lib/firewall.js check 203.0.113.10 9898 mirror.example.com 2003
node lib/firewall.js close 203.0.113.10 9898 mirror.example.com 2003
```

> **NOTE**
> Geofencing may contact the configured external HTTP geolocation endpoint. Consider privacy, egress controls, and dependency availability before enabling it.

11.2 Geolocation firewall rules

Geolocation is configured on the mirror in `conf/mirror.json`; there is no separate firewall CLI command for it.

```
{
  "firewall_geofencing_additive": ["ML", "SG", "JP", "private_blocks", "localhost"],
  "firewall_geofencing_subtractive": ["RU", "CN"],
  "firewall_geo_server": "www.geoplugin.net",
  "firewall_ip_cache_size": 2000000
}
```

- `firewall_geofencing_additive` — automatically allows clients whose location matches an entry, even if no explicit open command was issued.
  Valid country entries are ISO two-letter country codes, such as "IN", "SG", or "US".
- `firewall_geofencing_subtractive` — always denies matching client locations. This takes precedence over an explicit open firewall rule.
- `firewall_geo_server` — hostname used to look up public client IP locations. The current implementation calls it over HTTP on port 80.
- `firewall_ip_cache_size` — maximum number of IP-to-location lookups retained in memory.

Example: allow India, Singapore, and private-network addresses by default, but block Russia and China:

```
{
  "firewall_enabled_tcp": true,
  "firewall_geofencing_additive": ["IN", "SG", "private_blocks", "localhost"],
  "firewall_geofencing_subtractive": ["RU", "CN"],
  "firewall_geo_server": "www.geoplugin.net"
}
```

NOTE
Restart the mirror after changing these settings. If the geolocation service cannot be reached, an unknown public IP is not automatically allowed by the additive list; use open for explicit access where needed.

12 Production service management

Run each long-lived command under systemd, a container supervisor, or an equivalent process manager. Reflection intentionally exits when it loses connectivity, misses required heartbeats, or detects an address change; a supervisor must restart it.

13 Operation and validation checklist

- ☐ Check that the mirror process is running and listening on the configured control port.
- ☐ Check that each reflector process is running and that its log shows an active mirror listener and pool/channel activity.
- ☐ Test one application request through every published service port from the intended client network.
- ☐ Review `logs/*.ndjson` for [access], connection errors, heartbeat failures, and firewall blocks.
- ☐ Check disk usage under `logs/`, particularly if traffic capture is enabled.
- ☐ After any secret or configuration change, restart the mirror and reflector together and repeat the end-to-end test.

Useful checks:

```
ss -lntup
tail -f /opt/reflection/logs/mirror_tcp.log.ndjson
tail -f /opt/reflection/logs/reflector_tcp.log.ndjson
```

14 Troubleshooting

Table 6. Symptoms, likely causes, and corrective action.

Symptom	Likely cause and action
Reflector cannot start listener	Verify mirror hostname/port, network ACLs, matching `crypt.json`, and that the requested published port is free.
Mirror says no reflector is available	The reflector is down, cannot reach the mirror, has a key mismatch, or all TCP pool channels are allocated. Check reflector logs and raise `pool_size` if concurrency is expected.
Listener disappears	The mirror closes a listener when its last reflector channel closes. Restore the reflector; supervisor restart should recreate it.
Heartbeat exits a reflector	Verify the mirror heartbeat port and matching heartbeat key. Check `heartbeat_check_interval`, route/firewall, and the failure threshold.
Traffic reaches mirror but not origin	From the reflector host, test `reflected_host:reflected_port` directly. Verify DNS, routing, and origin ACLs.
UDP has no response	Check UDP ACLs in both directions, IPv4/IPv6 fields, cleanup timeout, and the TCP heartbeat reachability note.
Firewall blocks clients	Confirm the firewall feature is enabled intentionally, the client source IP is correct (including NAT), and send an open command to the trusted control port.
Logs grow quickly	Disable traffic logger/recorder, reduce debug logging, rotate `logs/`, and review data-retention obligations.
Service continuously restarts	Inspect the application log first. Common causes are unavailable mirror/origin, interface changes with `dhcp_check`, zero TCP channels beyond the configured wait, and conflicting ports.

15 Upgrade and rollback

Reflection is installed as a directory tree with no package manager step, so an upgrade is a file replacement and a rollback is a file restore. Plan both as a maintenance window: the published service port is unavailable while the mirror and reflector are stopped.

15.1 Before an upgrade

1. Record which launch commands are in service (Table 2) and which `conf/` files each host uses.
2. Copy the installation directory, for example `/opt/reflection`, to a dated location and keep it until the release is accepted.
3. Run the checks in section 13 before stopping anything, so that a later failure can be attributed to the change.
4. Confirm that `key` and `heartbeat_key` are held in the service manager environment, not only in `conf/crypt.json`.

> **WARNING**
> Upgrade the mirror and the reflector in the same window. Both ends must run matching `conf/crypt.json` values; a partial upgrade can present as the key mismatch described in section 14.

15.2 Upgrade

1. Stop the reflector first, then the mirror — the reverse of the start order in section 7.3.
2. Replace the program files with the new release and keep the in-service `conf/` directory.
3. Compare the `conf/` files supplied with the release against the in-service files and carry across only keys that are new. Never overwrite in-service values with the sample values described in section 6.
4. Restore the ownership and permissions set out in section 5.
5. Start the mirror first, then the reflector, as in section 7.3.
6. Work through section 13, including one real application request through every published service port.

15.3 Rollback

1. Stop the reflector, then the mirror.
2. Restore the directory copied in section 15.1, including its `conf/` directory.
3. Start the mirror, then the reflector, and work through section 13 again.
4. Record what failed, with the log extract that shows it, before retrying.

> **NOTE**
> A supervisor that restarts Reflection automatically (section 12) will restart it during the window. Mask the service unit for the duration and re-enable it once the end-to-end test has passed.

16 Handover acceptance record

Complete one record for each installed pair of hosts. It confirms that the deployment steps in Part I and the operational controls in Part II were carried out and verified on site.

Table 7. Handover acceptance record.

Item	Recorded value / confirmation
Site and system name	
Mirror host and Reflection control port	
Mirror heartbeat port	
Reflector host and installation directory	
Origin service host and port	
Published service port(s)	
Launch mode in service (TCP, UDP, or many TCP mappings)	
conf/crypt.json secrets replaced with site values	
Traffic logger and recorder disabled unless approved	
Firewall control port restricted to trusted administrators	
Supervisor configured to restart mirror and reflector	
Log rotation and retention configured	
End-to-end test through every published port passed on (date)	
Release and document version handed over	

Table 8. Handover signatures.

	Delivered by (Tekmonks)	Accepted by (customer)
Name		
Role		
Signature		
Date		

Appendix A Quick reference

Where each value used in this manual is defined. Section numbers refer to the two parts of this handover set.

Table 9. Where each configured value is defined.

Item	Where it is defined
Reflection control/data port	port in conf/mirror.json; mirror_port in conf/reflect.json (7.1, 7.2)
Heartbeat port	heartbeat_tcp in conf/mirror.json; mirror_heartbeat_tcp in conf/reflect.json (7.1, 7.2)
Published client-facing port	mirror_service_host and mirror_service_port in conf/reflect.json (7.2)
Origin service address	reflected_host and reflected_port in conf/reflect.json (7.2)
Multiple published mappings	reflected_hosts in conf/massreflect.json (8.1)
Shared secrets	key and heartbeat_key in conf/crypt.json, identical on both hosts (3, 6)
AES extension key	conf/aes.json, used by extensions/aes.js (6, 10)
Extension chain	conf/extensions.json (10)
Firewall admission control	firewall_* keys in conf/mirror.json; commands in lib/firewall.js (11)
Worker counts	cluster_mirror.json and cluster_reflector.json (9.3)
Logging	log.json; output written under logs/*.ndjson (9.3)

Appendix B Document control

Table 10. Document control.

Field	Value
Document	Reflection Administrator & Operations Manual
Part I	Deployment & Installation Guide (sections 1 to 8)
Part II	Administrator & Operations Manual (sections 9 to 16)
Version	1.0
Issued by	Tekmonks

Reflection · Administrator & Operations Manual · Part II · Version 1.0